

My Database Investigation Report

A Beginner's Look at Oracle, SQL Server, PostgreSQL and MySQL

Submitted by: Computer Science Student | June 2026

Introduction

Before this module, I thought a database was just a fancy spreadsheet. I had used MySQL once for a school project but I did not really understand why there were so many different database systems or why big companies paid so much money for them when free options exist.

After doing this investigation, I now understand that choosing a database is not just a technical decision — it is also about money, risk, regulations, and the history of the company. This report summarises what I learned.

1. The Four Main Database Systems

I investigated four relational database systems. They all store data in tables and support SQL, but they are very different under the hood.

Database	Owner	Cost	Best Known For
Oracle	Oracle Corp	Very expensive (~\$47,500 per 2 cores)	Big banks and governments
SQL Server	Microsoft	Expensive (~\$15,000 per 2 cores)	Companies using Microsoft tools
PostgreSQL	Community	Free forever (open source)	Developers and startups
MySQL	Oracle Corp	Free (community) or paid (enterprise)	Websites and web apps



All four databases do the same basic things: tables, joins, transactions, stored procedures. But the way they do it internally — and who they are designed for — is completely different.

2. How They Are Different

Scalability — What happens when more users join?

Scalability means: can the database keep up when more people use it? There are two ways to scale:

- Scale Up — buy a bigger, more powerful server
- Scale Out — add more servers and spread the load

Oracle has something called RAC (Real Application Clusters). This lets multiple servers all connect to the same database at the same time. If one server breaks, the others keep working. I found this impressive because other databases cannot do this out of the box.

PostgreSQL and MySQL can add read replicas — extra servers that handle read queries — but only one server can handle writes. This works well for most apps but has limits for huge systems like banks.

Security — Who can see the data?

Security was the topic that surprised me most. I thought all databases had the same basic security, but Oracle has features that no other system offers for free:

- Oracle Database Vault — even the database admin cannot read the data. This is important for banks where employees could steal customer information.
- Oracle Audit Vault — keeps a record of who accessed what and when. Regulators like banks to have this.
- Transparent Data Encryption (TDE) — encrypts all data on disk. Oracle includes this as a paid option; PostgreSQL has it through extensions for free.



PostgreSQL actually has quite good security too — it has Row Level Security built in for free, which lets you show different rows to different users. Oracle charges extra for this level of control.

Performance — How fast is it?

Performance depends heavily on what kind of work you are doing. I learned there are two main types:

- OLTP (Online Transaction Processing) — lots of small, fast transactions. Like a bank processing payments. Oracle, SQL Server and PostgreSQL all handle this well.
- OLAP (Analytical Processing) — big complex queries on millions of rows. Like a manager running a monthly sales report. Oracle Exadata is specially built for this.

One thing I found interesting: Oracle has an In-Memory Column Store that lets it do both OLTP and analytics from the same database. Other systems usually need a separate analytics database.

Licensing and Cost

This section honestly shocked me. I did not realise how expensive enterprise databases could be. Here is a simple example I worked out:

Scenario	Oracle EE	PostgreSQL
Licence (32 cores)	~\$760,000	\$0
Annual Support	~\$167,000/year	\$0 (community)
5-Year Total	~\$1,600,000+	\$0
DBA Needed?	Yes — Oracle specialist	Yes — but any dev can learn

After seeing these numbers I thought: why would anyone pay for Oracle? But then I read more and understood the reason — which I explain in Section 4.

Cloud Readiness

All four databases now run in the cloud, but in different ways:

- PostgreSQL — runs on AWS, Google Cloud, and Azure. All three cloud providers have built faster, improved versions of PostgreSQL on top of the open source code. It is the most flexible.
- SQL Server — works best on Microsoft Azure. There is a service called Azure SQL that manages everything for you.
- Oracle — prefers its own cloud (Oracle Cloud Infrastructure). If you run Oracle on AWS or Azure, they charge you more money. I think this is a clever business strategy.
- MySQL — Amazon built a faster version called Aurora MySQL which is very popular.

Backup and Recovery

I used to think backup just meant copying files. The reality for enterprise databases is much more complex:

- RPO (Recovery Point Objective) — how much data can you afford to lose? A bank's answer is zero.
- RTO (Recovery Time Objective) — how fast must the system come back online? A bank might say 30 seconds.

Oracle has a feature called Flashback that lets you travel back in time to any point and recover data without restoring from a backup. I think this is one of Oracle's coolest features. PostgreSQL achieves similar results using WAL (Write-Ahead Log) archiving, which is also very reliable but requires more setup.

3. Who Uses What?

This was one of the most interesting parts of my investigation. Different industries make very different choices:

Industry	Database Used	Main Reason
Banks	Oracle EE	Zero data loss, regulatory compliance, 24/7 uptime with RAC
Government	Oracle / SQL Server	Long procurement cycles, established vendor relationships
Telecom	Oracle EE	Billions of call records, partitioning, Exadata analytics
Startups	PostgreSQL	Free, easy to use, great cloud options like Supabase
Web / Apps	MySQL	Powers WordPress, Shopify, and most of the internet
Cloud companies	PostgreSQL	No licence cost, works with all cloud providers
Microsoft shops	SQL Server	Already use Windows, Azure, Power BI, and .NET

4. Why Do Companies Pay Millions for Oracle?

This was the question I kept asking myself during the investigation. PostgreSQL is free, reliable, and used by companies like Instagram and Notion. So why do banks pay Oracle millions of dollars every year?

After researching, I found several reasons that actually make sense:

The cost of failure is bigger than the cost of Oracle

A large bank might lose \$10 million or more for every hour the database is down. If Oracle prevents even one major outage per year, the licence cost pays for itself many times over. From this angle, Oracle is actually cheap.

Regulators recognise Oracle

When a bank is audited by a regulator, saying 'we use Oracle Enterprise Edition with Advanced Security and Audit Vault' is an answer the auditor understands and accepts. Saying 'we use PostgreSQL with some security extensions we configured ourselves' creates questions. This might seem unfair, but it is the reality of regulated industries.

Switching is incredibly expensive

A bank that has used Oracle for 20 years has thousands of stored procedures written in PL/SQL (Oracle's programming language), Oracle-specific reporting tools, certified core banking software that only runs on Oracle, and a team of Oracle-certified database administrators. Switching to PostgreSQL would mean

rewriting all of that. The cost could be hundreds of millions and take 5 to 10 years. It is often cheaper to just keep paying Oracle.

Support matters more than the software

If something breaks at 3am on a Sunday, PostgreSQL's community forum is not enough for a bank. Oracle's support contract means an Oracle engineer calls you back within 30 minutes. That accountability is worth paying for when the alternative is a major financial incident.

 This taught me that choosing a database is not just about which one is technically best. It is about risk, accountability, switching costs, and organisational history. The 'best' database is always the right one for that specific situation.

5. Why Not Just Use One Database for Everything?

I asked myself this question a lot. If they all support SQL, tables, and transactions — why have four different ones?

The answer comes down to three main reasons:

Different problems need different solutions

A bank doing millions of transactions per second has completely different needs from a startup building a mobile app. A mapping application needs geospatial features (PostgreSQL with PostGIS is great for this). A news website needs fast, simple reads (MySQL is fine). You cannot optimise one database for all these situations at once.

The CAP Theorem

I learned about something called the CAP Theorem, which says a distributed database can only guarantee two of three things: Consistency (data is always correct), Availability (system always responds), and Partition Tolerance (works even if the network breaks). Since you cannot have all three, different databases make different trade-offs — and that means different databases are better for different situations.

History and money keep old choices alive

Many companies are still using the database they chose 15 or 20 years ago — not because it is the best option today, but because switching costs too much. This means the database market naturally has many different systems in use at the same time.

6. What I Learned From This Investigation

When I started this investigation, I thought I would just be comparing features in a table. Instead I ended up learning a lot about how businesses and technology actually work together.

- Databases are not just technical tools — they are business decisions tied to risk, money, regulation, and company culture.
- Free does not always mean cheap. PostgreSQL is free to download, but running it properly at enterprise scale still needs skilled people and good infrastructure.
- Oracle is expensive for a reason. For companies where failure is unacceptably costly, the price is justified.
- PostgreSQL is growing fast. More and more companies are migrating away from Oracle to save money, and cloud providers are building powerful tools on top of it.
- The 'best' database is always the right one for the situation — not the most powerful or the most popular.



If I were starting a new project today, I would use PostgreSQL. It is free, powerful, supported by all cloud providers, and has a huge community. But if I were advising a large bank, I would understand why they use Oracle — and I would not recommend changing it without a very strong reason.

Conclusion

This investigation gave me a much deeper understanding of database management systems than I expected. I came in thinking it was mostly about SQL syntax and features. I came out understanding that the real questions are about risk tolerance, vendor relationships, regulatory requirements, and the cost of change.

The most important thing I learned is that in real enterprise environments, technical decisions are almost never purely technical. They involve business strategy, financial planning, legal compliance, and organisational history. Understanding this is what separates a software engineer who writes code from one who designs systems.

Word count: approximately 1,400 words | Submitted: June 2026